Suman Sourav
Jonathan J. Huerta y Munive

AALBORG UNIVERSITY
DENMARK

Models and Tools for Cyber-Physical Systems
Exercise sheet 5
WITH SOLUTIONS

Exercise 1: Asynchronous process `Split`

We want to design an asynchronous process `Split` that is the dual of `Merge` (see Figure 1 or Chapter 4, Slide 6). The process `Split` has one input channel `in` and two output channels `out1` and `out2`. The messages received on the input channel should be routed to one of the output channels in a nondeterministic manner so that all possible splittings of the input stream are feasible executions. One example execution is:

$$\texttt{in}: 1\ 2\ 3\ 4\ 5\ 6\ \ldots \quad \rightsquigarrow \quad \begin{array}{lcccccccc} \texttt{out1}: & 1 & & 3 & & & 6 & \ldots \\ \texttt{out2}: & & 2 & & 4 & 5 & & \ldots \end{array}$$

(Read as: first output "1" on `out1`, then output "2" on `out2`, etc.)

Describe all parts of the desired process `Split`.

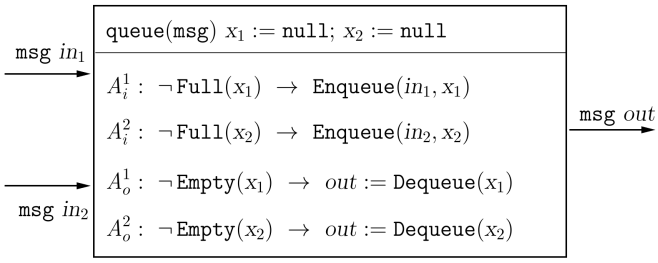

Figure 1: Asynchronous process `Merge`.

. . . . . Solution sketch . . . . . .

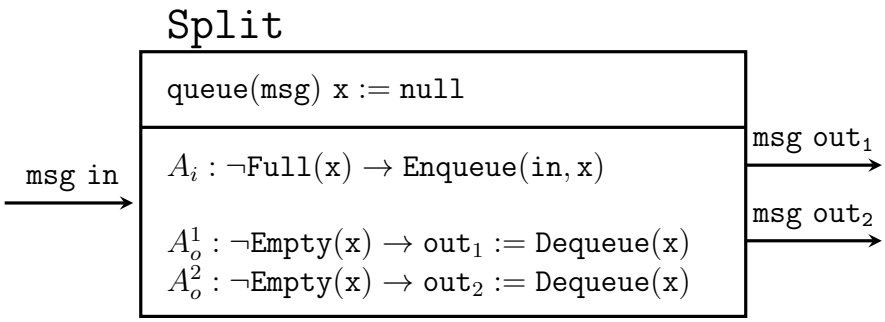

Figure 2: Asynchronous process `Split`.

Figure 2 shows the block diagram. Below, we describe each part individually.

The asynchronous process Split has a single input channel in of type msg. The output channels are $\mathtt{out}_1$ and $\mathtt{out}_2$ of type msg. There is a single queue as state variable with the declaration and initialization given by:

$$\mathtt{queue}(\mathtt{msg})\ \mathtt{x} := \mathtt{null}$$

The input task A_i stores the messages arriving on the input channel in the queue x:

$$\neg\mathtt{Full}(\mathtt{x}) \rightarrow \mathtt{Enqueue}(\mathtt{in}, \mathtt{x})$$

The output task A_o^1 is enabled when the queue x is nonempty; a message is removed from the queue and transmitted on the output channel $\mathtt{out}_1$:

$$\neg\mathtt{Empty}(\mathtt{x}) \rightarrow \mathtt{out}_1 := \mathtt{Dequeue}(\mathtt{x}).$$

The output task A_o^2 is symmetric and transmits messages on the output channel $\mathtt{out}_2$:

$$\neg\mathtt{Empty}(\mathtt{x}) \rightarrow \mathtt{out}_2 := \mathtt{Dequeue}(\mathtt{x}).$$

Note that a message stored in the queue x is transmitted on only one of the output channels, and the choice is nondeterministic.

Exercise 2: Asynchronous process composition

The asynchronous process DoubleBuffer is the result of the parallel composition of two Buffer components. In Chapter 4, Slide 16 we have seen the block diagram of the DoubleBuffer process. In the following, consider the asynchronous process

$$(\ \mathtt{Merge}[\mathtt{out} \mapsto \mathtt{temp}] \mid \mathtt{Merge}[\mathtt{in}_1 \mapsto \mathtt{temp}][\mathtt{in}_2 \mapsto \mathtt{in}_3]\ ) \setminus \mathtt{temp}$$

obtained by composing two instances of the process Merge (see Figure 1). Construct the block diagram of this parallel composition and explain the input/output behavior.

.....Solution sketch .....

Figure 3 shows the block diagram. Below, we describe each part individually.

The input channels are $\mathtt{in}_1$, $\mathtt{in}_2$, and $\mathtt{in}_3$, all of type msg. The output channel is out. When composing the two instances of Merge, we need to make sure that the state variables have distinct names. The state variables of the composition and their initialization are specified by:

$$\mathtt{queue}(\mathtt{msg})\ \mathtt{x}_1 := \mathtt{null}; \mathtt{x}_2 := \mathtt{null}; \mathtt{y}_1 := \mathtt{null}; \mathtt{y}_2 := \mathtt{null}$$

The composition has three input tasks corresponding to its three input channels specified by:

$$\begin{array}{ll} A_i^1: & \neg\mathtt{Full}(\mathtt{x}_1) \rightarrow \mathtt{Enqueue}(\mathtt{in}_1, \mathtt{x}_1) \\ A_i^2: & \neg\mathtt{Full}(\mathtt{x}_2) \rightarrow \mathtt{Enqueue}(\mathtt{in}_2, \mathtt{x}_2) \\ A_i^3: & \neg\mathtt{Full}(\mathtt{y}_2) \rightarrow \mathtt{Enqueue}(\mathtt{in}_3, \mathtt{y}_2) \end{array}$$

The composition has two internal tasks, each of which is obtained by synchronizing, via the channel temp, an output task of the first instance with a corresponding input task of the second instance:

$$\begin{array}{ll} A^1: & \neg\mathtt{Empty}(\mathtt{x}_1) \wedge \neg\mathtt{Full}(\mathtt{y}_1) \rightarrow \\ & \{\mathtt{local\ msg\ temp} := \mathtt{Dequeue}(\mathtt{x}_1);\ \mathtt{Enqueue}(\mathtt{temp}, \mathtt{y}_1)\} \\ A^2: & \neg\mathtt{Empty}(\mathtt{x}_2) \wedge \neg\mathtt{Full}(\mathtt{y}_1) \rightarrow \\ & \{\mathtt{local\ msg\ temp} := \mathtt{Dequeue}(\mathtt{x}_2);\ \mathtt{Enqueue}(\mathtt{temp}, \mathtt{y}_1)\} \end{array}$$

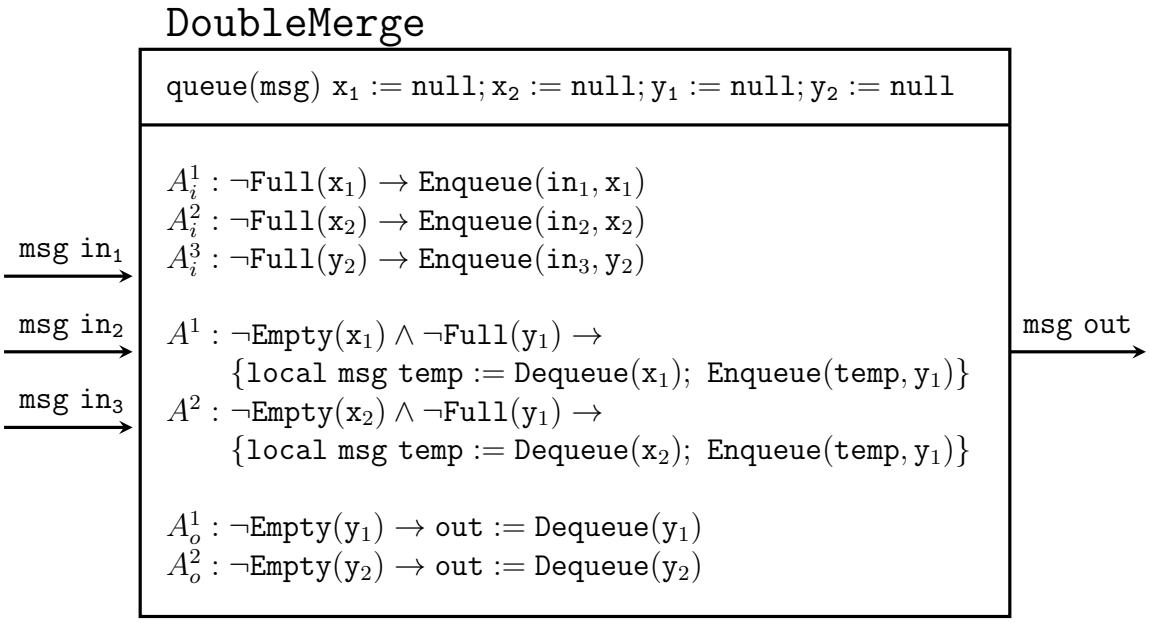

Figure 3: Asynchronous process `DoubleMerge`.

Finally, the composition has two output tasks that remove messages from the queues y_1 and y_2 in order to transmit them on the output channel out:

$$\begin{array}{ll} A_o^1 : & \neg\texttt{Empty}(y_1) \rightarrow \texttt{out} := \texttt{Dequeue}(y_1) \\ A_o^2 : & \neg\texttt{Empty}(y_2) \rightarrow \texttt{out} := \texttt{Dequeue}(y_2) \end{array}$$

The sequence of values output by the composition represents a merge of the sequences of input values received on the three input channels. The relative order of values received on each of the input channels is preserved in the output sequence, but the three input sequences can be interleaved in any order (nondeterministically).

Exercise 3: Peterson's mutual exclusion protocol (Uppaal)
Download the Uppaal model for Peterson's mutual exclusion protocol from Moodle. The model includes the component `UnfairScheduler`, which emulates the scheduler of an operating system.

(a) Check whether the protocol satisfies the mutual exclusion property:

$$\forall\Box\ \neg(\texttt{P1.Crit} \wedge \texttt{P2.Crit})$$

(b) Check whether the protocol satisfies the non-starvation property:
"If process 1 wants to enter its critical section, then it eventually does."

$$\forall\Box\ (\texttt{P1.Try1} \rightarrow \forall\Diamond\ \texttt{P1.Crit})$$

Uppaal has a dedicated syntax for this pattern: `P1.Try1 --> P1.Crit`

(c) Build a component `FairScheduler` such that the above properties are satisfied.

Solution sketch

Download the UPPAAL model solution from Moodle.

The idea of this simple solution is to let the scheduler alternate between the two processes.